Experiment No. 1

STUDY OF LOGIC GATES

AIM: To familiarize

i. TTL and CMOS Logic ICs for AND, OR, NOT, NAND, NOR and XOR by verification of truth tables.

COMPONENTS AND EQUIPMENTS REQUIRED:

SL No.	COMPONENTS	SPECIFICATION	QTY
1.	AND GATE	IC 7408	1
2.	OR GATE	IC 7432	1
3.	NOT GATE	IC 7404	1
4.	NAND GATE 2 I/P	IC 7400	1
5.	NOR GATE	IC 7402	1
6.	X-OR GATE	IC 7486	1
7.	IC TRAINER KIT	-	1
8.	Connecting Wires	As required	

PRINCIPLE:

Circuit that takes the logical decision and the process are called logic gates. Each gate has one or more input and only one output. OR, AND and NOT are basic gates

AND GATE:

The AND gate performs a logical multiplication commonly known as AND function. The output is high when both the inputs are high. The output is low level when any one of the inputs is low. Output Y= A.B where A and B are the inputs.

OR GATE:

The OR gate performs a logical addition commonly known as OR function. The output is high when any one of the inputs is high. The output is low level when both the inputs are low. Output Y= A+B where A and B are the inputs.

NOT GATE:

The NOT gate is called an inverter. The output is high when the input is low. The output is low when the input is high. Output Y= NOT(A) where A is the input.

NAND GATE:

The NAND gate is a combination of AND-NOT. The output is high when any one of the input is low or both inputs are low .The output is low level only when both inputs are high.

NOR GATE:

The NOR gate is a combination of OR-NOT. The output is high when both inputs are low. The output is low when one or both inputs are high.

X-OR GATE:

The output is high when any one of the inputs is high. The output is low when both the inputs are low and both the inputs are high.

PROCEDURE:

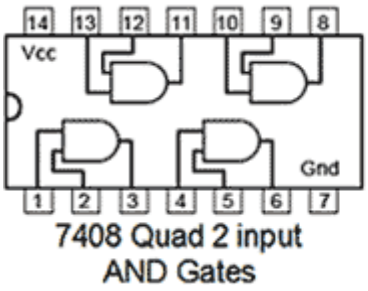
1. Check the all the required components.
2. Connect the circuit as per the circuit diagram.
3. Apply Vcc and Ground.
4. Logical inputs are given as per truth table using switches provided.
5. Observe the output on LEDs and verify the truth table.

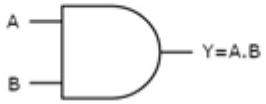
RESULT:

Familiarized with basic TTL and CMOS logic gate ICs and verified their truth tables.

AND GATE

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**

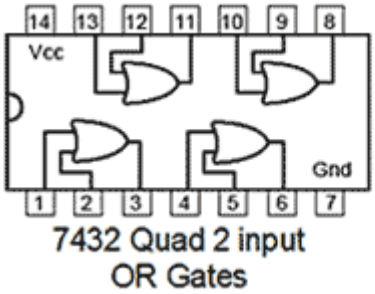


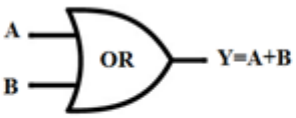


A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

OR GATE:

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**

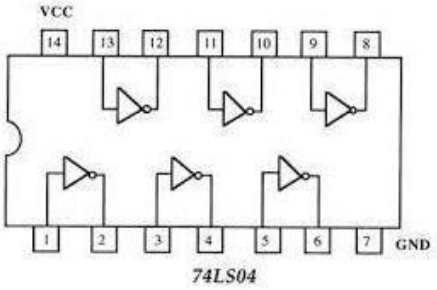


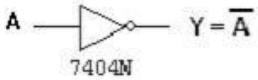


A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT GATE:

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**

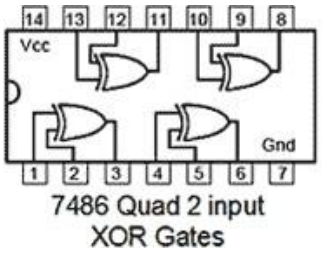




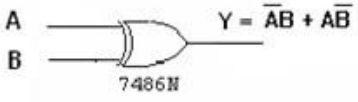
A	$\overline{A}$
0	1
1	0

X-OR GATE :

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**



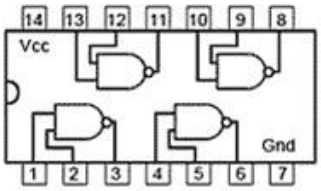
7486 Quad 2 input
XOR Gates



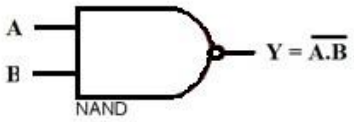
A	B	$\overline{A}B + A\overline{B}$
0	0	0
0	1	1
1	0	1
1	1	0

2-INPUT NAND GATE:

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**



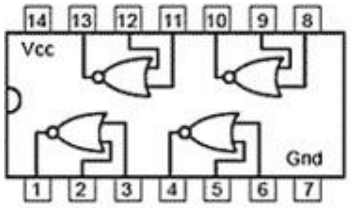
7400 Quad 2 input
NAND Gates



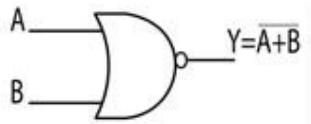
A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE:

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**



7402 Quad 2 input
NOR Gates



A	B	Y= $\overline{A+B}$
0	0	1
0	1	1
1	0	1
1	1	0

Experiment No. 2

NAND AS UNIVERSAL GATE

AIM:

To familiarize Universal gates for implementing other logic functions.

OBJECTIVES:

- They will understand the universal property of NAND gate.

COMPONENTS AND EQUIPMENTS REQUIRED:

SL No.	COMPONENTS	SPECIFICATION	QTY
1.	NAND GATE 2 I/P	IC 7400	1

PRINCIPLE:

A universal gate is a gate which can implement any Boolean function without need to use any other gate type. The NAND and NOR gates are universal gates. NAND and NOR gates are economical and easier to fabricate and are the basic gates used in all IC digital logic families.

PROCEDURE:

1. Check the all the required components.
2. Connect the circuit as per the circuit diagram.
3. Apply Vcc and Ground.
4. Logical inputs are given as per truth table using switches provided.
5. Observe the output on LEDs and verify the truth table.

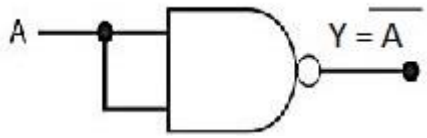
RESULT:

Constructed basic logic gates and X-OR using NAND gate.

INFERENCE:

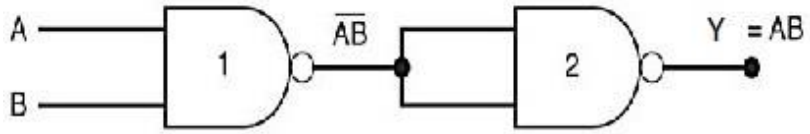
1. By using NAND gate we can construct any logic circuits.

a) NOT gate using NAND gate



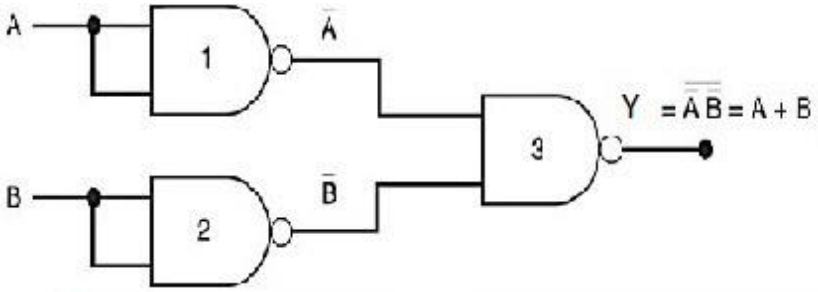
A	$\overline{A}$
0	1
1	0

b) AND gate using NAND gate



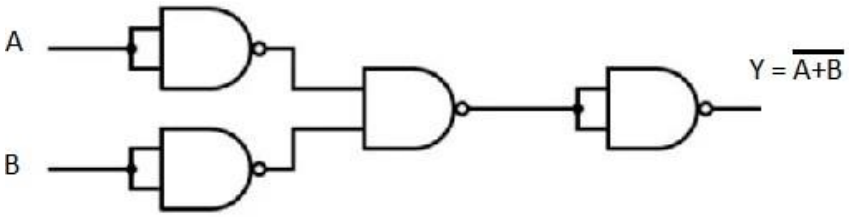
A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

c) OR gate using NAND gate



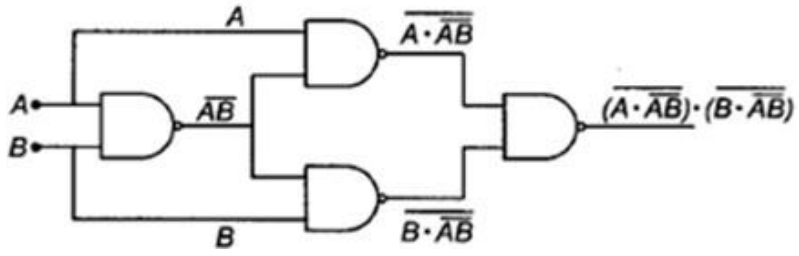
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

d) NOR gate using NAND gate



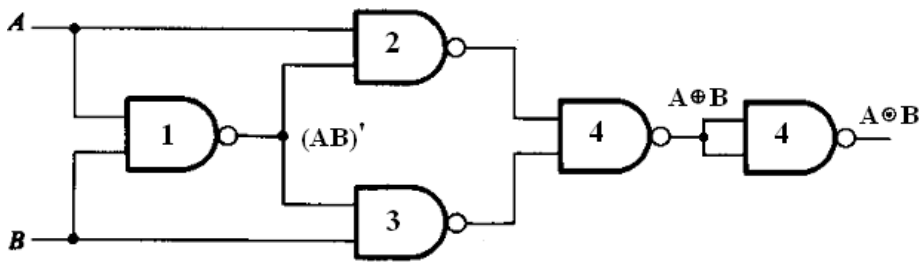
A	B	Y= $\overline{A+B}$
0	0	1
0	1	1
1	0	1
1	1	0

e) XOR gate using NAND gate



A	B	$\overline{AB}$ + $\overline{AB}$
0	0	0
0	1	1
1	0	1
1	1	0

f) XNOR gate using NAND gate



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Experiment No. 3

NOR AS UNIVERSAL GATE

AIM:

To familiarize Universal gates for implementing other logic functions.

OBJECTIVES:

- They will understand the universal property of NOR gate.

COMPONENTS AND EQUIPMENTS REQUIRED:

SL No.	COMPONENTS	SPECIFICATION	QTY
1.	NOR GATE	IC 7402	1

PRINCIPLE:

A universal gate is a gate which can implement any Boolean function without need to use any other gate type. The NAND and NOR gates are universal gates. NAND and NOR gates are economical and easier to fabricate and are the basic gates used in all IC digital logic families.

PROCEDURE:

1. Check the all the required components.
2. Connect the circuit as per the circuit diagram.
3. Apply Vcc and Ground.
4. Logical inputs are given as per truth table using switches provided.
5. Observe the output on LEDs and verify the truth table.

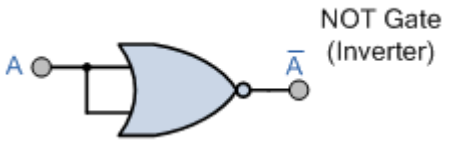
RESULT:

Constructed basic logic gates and X-OR using NOR gate.

INFERENCE:

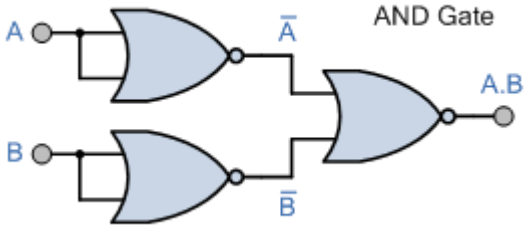
1. By using NOR gate we can construct any logic circuits.

a) NOT gate using NOR gate



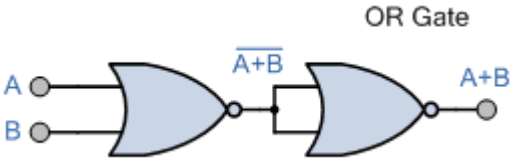
A	$\overline{A}$
0	1
1	0

b) AND gate using NOR gate



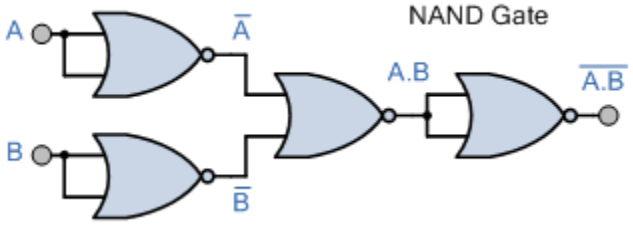
A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

c) OR gate using NOR gate



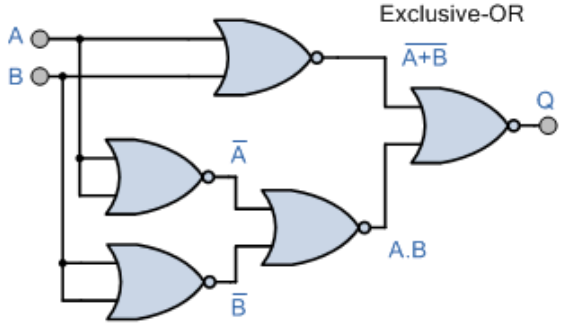
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

d) NAND gate using NOR gate



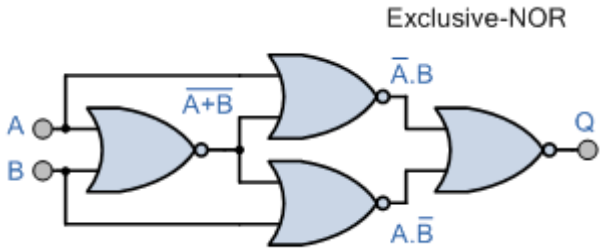
A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

e) XOR gate using NOR gate



A	B	$\overline{A}B + A\overline{B}$
0	0	0
0	1	1
1	0	1
1	1	0

f) XNOR gate using NOR gate



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Experiment No. 4

HALF AND FULL ADDERS

AIM:

To construct half adder and full adder circuits and verify the truth table using logic gates.

COMPONENTS AND EQUIPMENTS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1.	AND GATE	IC 7408	1
2.	X-OR GATE	IC 7486	1
3.	OR GATE	IC 7432	1
4.	IC TRAINER KIT	-	1
5.	Connecting wires	as required	

PRINCIPLE:

HALF ADDER:

A half adder has two inputs for the two bits to be added and two outputs one from the sum S and other from the carry C. The Sum is obtained from the X-OR Gate and the Carry out from the AND gate.

FULL ADDER:

A full adder is a combinational circuit that forms the arithmetic sum of three inputs and generates two outputs. A full adder is useful to add three bits at a time but a half adder cannot do so.

Design

HALF ADDER

TRUTH TABLE:

A	B	CARRY	SUM
0	**0**	**0**	**0**
0	**1**	**0**	**1**
1	**0**	**0**	**1**
1	**1**	**1**	**0**

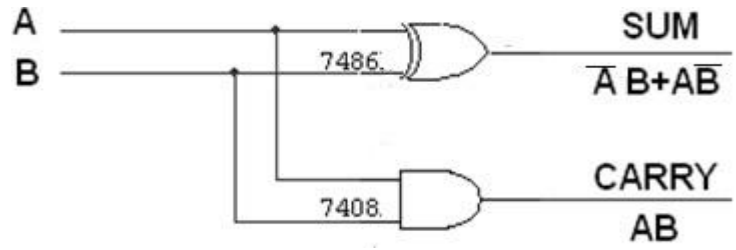
From above truth table, the expressions for carry and sum can be derived using the sum product terms.

$$\text{Sum} = \overline{A}.B + A.\overline{B}$$

$$= A \oplus B$$

Carry= AB

LOGIC DIAGRAM:



FULL ADDER

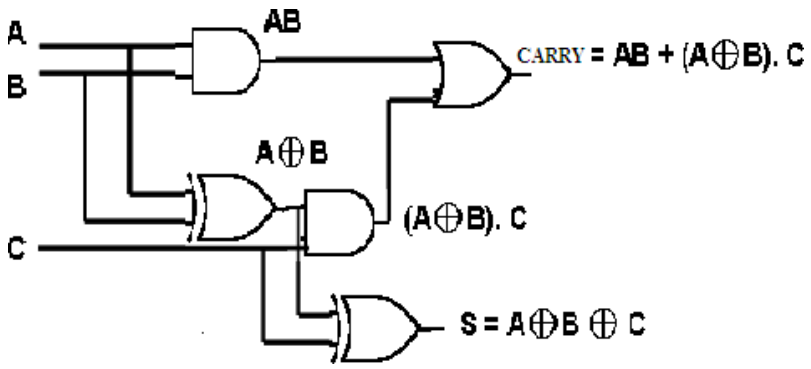
TRUTH TABLE:

A	B	C	CARRY	SUM
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

DESIGN:

$$\text{Sum} = \overline{A}\,\overline{B}\,C + \overline{A}\,B\overline{C} + A\overline{B}\,\overline{C} + ABC$$
$$= (\overline{A}\,\overline{B} + AB).C + (\overline{A}\,B + A\overline{B})\,\overline{C}$$
$$= \overline{(A \oplus B)}\,C + (A \oplus B)\,\overline{C}$$
$$= (A \oplus B) \oplus C$$
$$= \mathbf{(A \oplus B) \oplus C}$$

$$\text{CARRY} = \overline{A}\,BC + A\overline{B}\,C + AB\overline{C} + ABC$$
$$= (\overline{A}\,B + A\overline{B}).C + AB\,(\overline{C} + C)$$
$$= \mathbf{(A \oplus B).C + AB} \qquad \text{(we know that } \overline{C} + C = 1\text{)}$$



PROCEDURE:

1. Check the required components
2. Connections are given as per circuit diagram.
3. Logical inputs are given as per truth table.
4. Observe the output and verify the truth table.

RESULT:

Designed and constructed half adder and full adder, circuits and verified the truth table using logic gates.

Inference :

1. A Full adder can handle additional carry input compared to a Half adder.
2. Students get a basic idea about the addition circuits used in calculators and ALU.

Experiment No. 5

BINARY TO GRAY AND GRAY TO BINARY CONVERTER

AIM:

To construct and verify the truth tables of

(i) Binary to gray code converter

(ii) Gray to binary code converter

COMPONENTS AND EQUIPMENTS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1.	X-OR GATE	IC 7486	1
2.	IC TRAINER KIT	-	1
3.	Connecting wires	as required	

THEORY:

The availability of large variety of codes for the same discrete elements of information results in the use of different codes by different systems. A conversion circuit must be inserted between the two systems if each uses different codes for same information. Thus, code converter is a circuit that makes the two systems compatible even though each uses different binary code.

Here we take 4 bits for Gray to binary and vice versa conversions. Thus there are four inputs and four outputs. Gray code is a non-weighted code and its hamming distance is always 1.

The input variables for a Binary to Gray converter are designated as B3, B2, B1, and B0 and the output variables are designated as G3, G2, G1,and G0. From the truth table, combinational circuit is designed through K-Map.

Meanwhile the inputs for a Gray to Binary converter is G3, G2, G1,and G0 and its output generates B3, B2, B1, and B0.

DESIGN

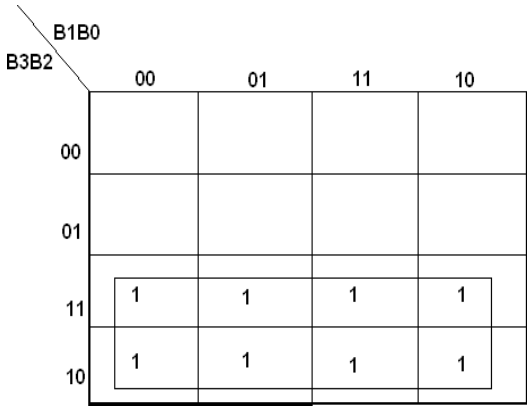
BINARY TO GRAY CODE CONVERTOR

TRUTH TABLE:

Binary inputs				**Gray outputs**			
B3	**B2**	**B1**	**B0**	**G3**	**G2**	**G1**	**G0**
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**
0	**0**	**0**	**1**	**0**	**0**	**0**	**1**
0	**0**	**1**	**0**	**0**	**0**	**1**	**1**
0	**0**	**1**	**1**	**0**	**0**	**1**	**0**
0	**1**	**0**	**0**	**0**	**1**	**1**	**0**
0	**1**	**0**	**1**	**0**	**1**	**1**	**1**
0	**1**	**1**	**0**	**0**	**1**	**0**	**1**
0	**1**	**1**	**1**	**0**	**1**	**0**	**0**
1	**0**	**0**	**0**	**1**	**1**	**0**	**0**

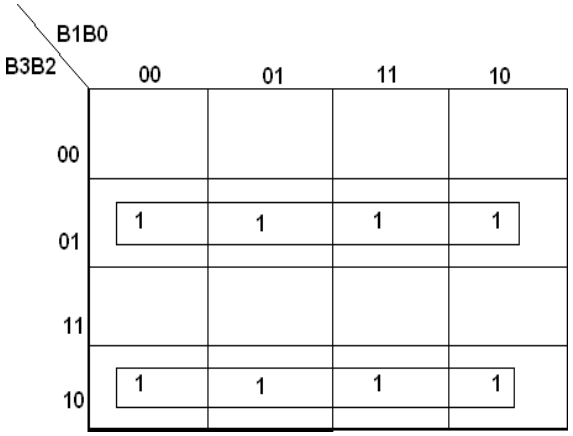
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

K-Map for G_3:



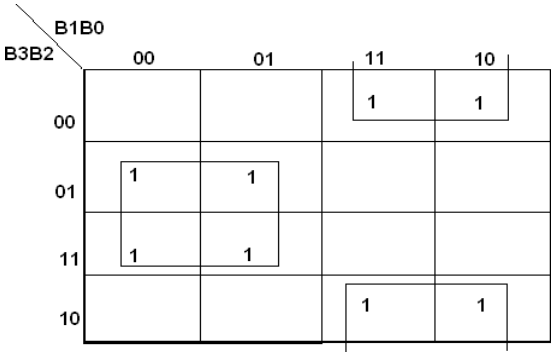
$G_3 = B_3$

K-Map for G_2:



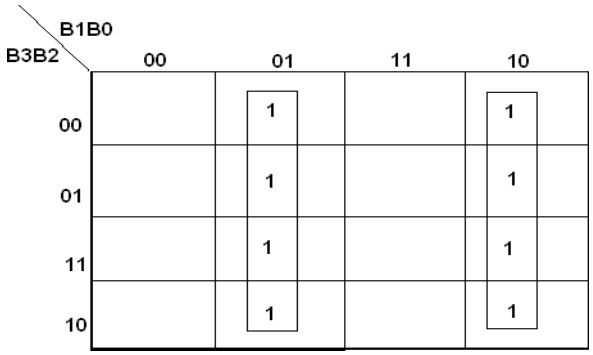
$G2 = \overline{B3}B2 + B3\overline{B2}$
$= B3 \oplus B2$

K-Map for G_1:



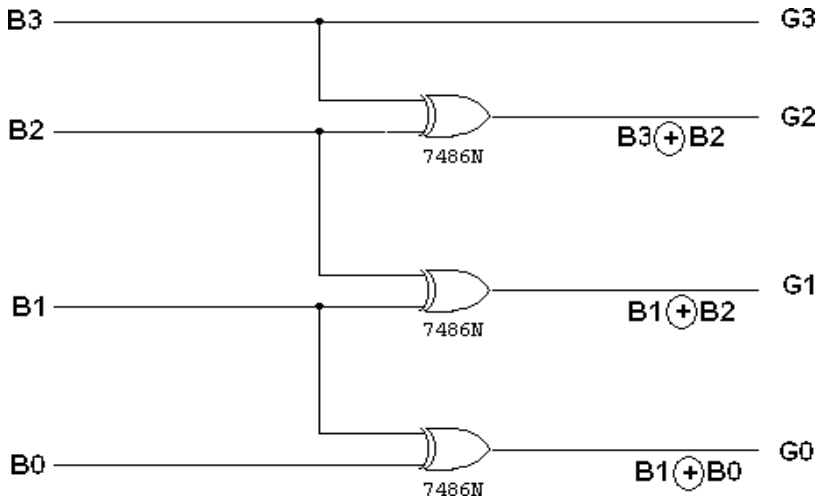
$G1 = \overline{B1}B2 + B1\overline{B2}$
$= B1 \oplus B2$

K-Map for G_0:



$$G0 = \overline{B1}B0 + B1\overline{B0}$$
$$= B1 \oplus B0$$

LOGIC DIAGRAM:-

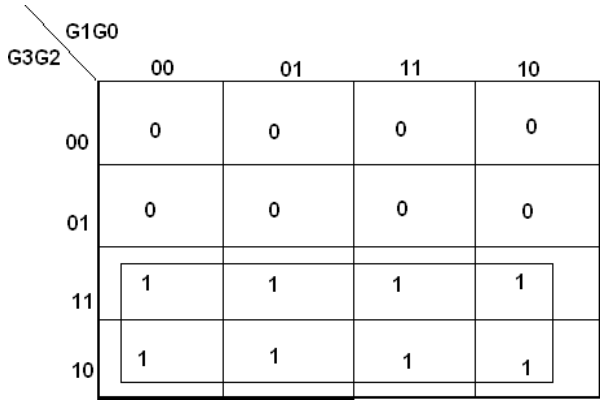


GRAY CODE TO BINARY CONVERTOR
TRUTH TABLE:

Gray Code				Binary Code			
G3	G2	G1	G0	B3	B2	B1	B0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	1	1	0	1	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1

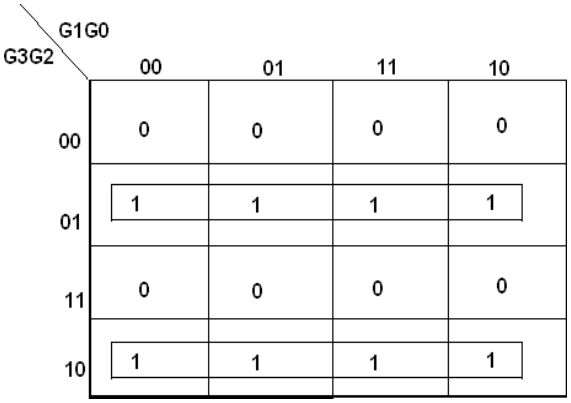
1 1	0 0	0 0	1 0	1 1	1 1	1 1	0 1

K-Map for B_3:



B3 = G3

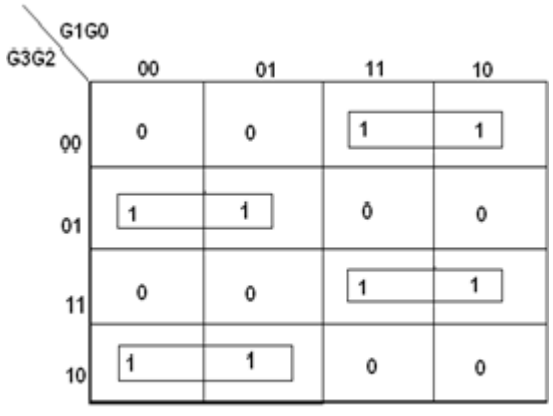
K-Map for B_2:



$B2 = \overline{G3}G2 + G3\overline{G2}$

$= G3 \oplus G2$

K-Map for B_1:



$B1 = \overline{G3}G2\overline{G1} + G3\overline{G2}\overline{G1} + \overline{G3}\overline{G2}G1 + G3G2G1$

$= \overline{G1}(\overline{G3}G2 + G3\overline{G2}) + G1(\overline{G3}\overline{G2} + G3G2)$

$= \overline{G1}.(G3 \oplus G2) + G1.\overline{(G3 \oplus G2)}$

$= G1 \oplus G2 \oplus G3$

$= G1 \oplus B2$

K-Map for B_0:

G3G2 \ G1G0	00	01	11	10
00	0	①	0	①
01	①	0	①	0
11	0	①	0	①
10	①	0	①	0

B0 = ----------

= ----------

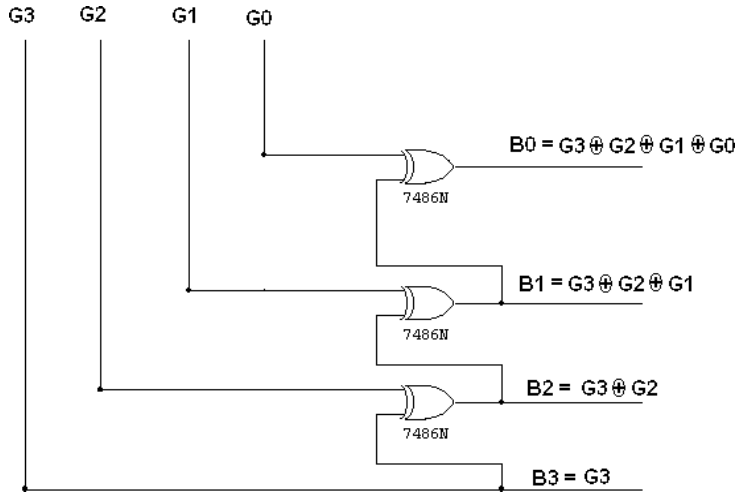
= ----------

= G3 ⊕ G2 ⊕ G1 ⊕ G0

= B1 ⊕ G0

(Solution should be completed by students)

LOGIC DIAGRAM OF GRAY TO BINARY CODE CONVERTER



PROCEDURE:

1. The circuit connections are made as shown in fig.
2. Pin (14) is connected to +Vcc and Pin (7) to ground.
3. In the case of binary to gray conversion, the inputs B0, B1, B2 and B3 are given at respective pins and outputs G0, G1, G2, G3 are taken for all the 16 combinations of the input.
4. In the case of gray to binary conversion, the inputs G0, G1, G2 and G3 are given at respective pins and outputs B0, B1, B2, and B3 are taken for all the 16 combinations of inputs.
5. The values of the outputs are tabulated.

RESULT:

Designed and implemented

a. Binary to gray code converter
b. Gray to binary code converter

Inference :

1. By using a single IC we can do both Gray to Binary and Binary to Gray converter.
2. Students get an idea to construct circuit for any code converter.

Experiment No. 6

MULTIPLEXER AND DEMULTIPLEXER

AIM:

To construct a 4:1 MUX and a 1: 4 DEMUX using logic gates.

OBJECTIVES: After completion of experiment

- To learn about various applications of multiplexer and de-multiplexer
- To learn and understand the working of MUX and DEMUX

.

COMPONENTS AND EQUIPMENTS:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1.	3 I/P AND GATE	IC 7411	2
2.	OR GATE	IC 7432	1
3.	NOT GATE	IC 7404	1
4.	IC TRAINER KIT	IC 7410	1
5	CONNECTING WIRES	As required	

PRINCIPLE:

Multiplexers are very useful components in digital systems. They transfer a large number of information units over a smaller number of channels or lines, (usually one line) under the control of selection signals. Multiplexer means many to one. A multiplexer is a circuit with many inputs but only one output. By using control signals (select lines) we can select any input to the output. Multiplexer is also called as data selector because the output bit depends on the input data bit that is selected. The general multiplexer circuit has 2^n input signals, n control/select signals and 1 output signal.

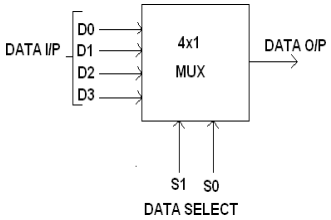
De-multiplexers perform the opposite function of multiplexers. They transfer a small number of information units over a larger number of channels under the control of selection signals. The general de-multiplexer circuit has 1 input signal, n control/select signals and 2n output signals. De-multiplexer circuit can also be realized using a decoder circuit with additional select/control inputs for the selection of output line into which the data is to be transmitted

PROCEDURE:

1. Test all the ICs manually/ using IC tester.
2. Connections are made as in the logic circuit diagram and give Vcc and the ground.
3. Connect the appropriate pins to the input switches and output LEDs.
4. Give various combinations of the inputs and observe the output and verify the truth table.
5. Similarly verify the demux circuit with truth table.

CIRCUIT DIAGRAM:

BLOCK DIAGRAM FOR 4:1 MULTIPLEXER:



DESIGN: 4:1 MUX FUNCTION TABLE:

SELECT INPUTS		DATA INPUTS				min term	OUTPUT Y
S1	S0	I0	I1	I2	I3		
0	0	1	x	x	x	$I0\bar{S1}\bar{S0}$	1
0	1	x	1	x	x	$I1\bar{S1}S0$	1
1	0	x	x	1	x	$I2S1\bar{S0}$	1
1	1	x	x	x	1	$I3S1S0$	1

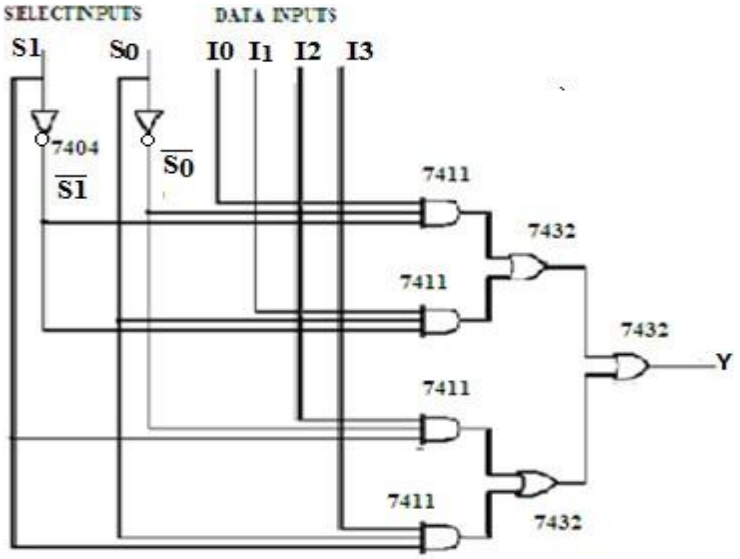
SOP of output $Y = I0\,\bar{S1}\,\bar{S0} + I1\,\bar{S1}S0 + I2\,S1\,\bar{S0} + I3\,S1\,S0$

4:1 MUX USING NAND GATES

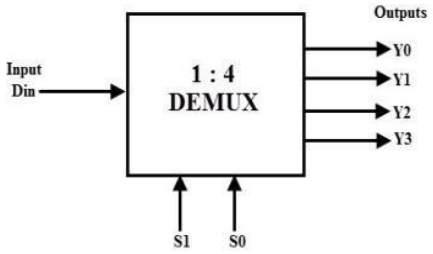
To implement using NAND gates, apply double inversion rule.

Then , $\overline{\overline{Y = I0\bar{S1}\bar{S0} + I1\bar{S1}S0 + I2S1\bar{S0} + I3S1S0}} = \overline{\overline{I0\bar{S1}\bar{S0}} \cdot \overline{I1\bar{S1}S0} \cdot \overline{I2S1\bar{S0}} \cdot \overline{I3S1S0}}$.

CIRCUIT DIAGRAM S: 1:4 MULTIPLEXER USING BASIC GATES:



BLOCK DIAGRAM FOR 1:4 DEMULTIPLEXER:



DESIGN

1:4 DEMUX FUNCTION TABLE

DATA INPUT	SELECT INPUTS		OUTPUTS			
	S1	S0	Y0	Y1	Y2	Y3
D	0	0	**D**	0	0	0
D	0	1	0	**D**	0	0
D	0	1	0	0	**D**	0
D	1	1	0	0	0	**D**

From the truth table, when S1=0 and S0= 0, the data input is connected to output Y0 and when S1= 0 and S0=1, then the data input is connected to output Y1. Similarly, other outputs are connected to the input for other two combinations of select lines.

From the table, the output logic can be expressed as min terms and are given below.

$Y0 = \overline{S1}\ \overline{S0}\ D$

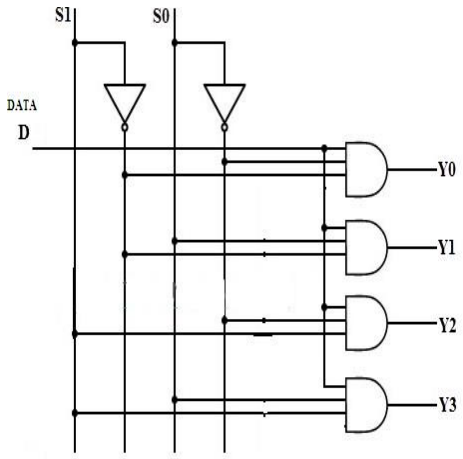
$Y1 = \overline{S1}\ S0\ D$

$Y2 = S1\ \overline{S0}\ D$

$Y3 = S1\ S0\ D$ Where D is the input data, Y0 to Y3 are output lines and S0 & S1 are select lines.

By applying double inversion rule to each output above, demux using NAND gates can be realized.

4:1 DEMUX USING BASIC GATES



RESULT:

Realized 4:1 Multiplexer and 1:4 Demultiplexer using basic gates and NAND gates and verified the truth table.

INFERENCE:

1. Many input data lines can be switched to a single line using multiplexer.

2. Data is directed to one of many output lines in a DEMUX circuit.

3. Selecting different IO devices for data transfer can be done using MUX.

4. These circuits are used for data routing.

Experiment No. 7

STUDY OF MULTPLEXER IC 74151 & DE MULTPLEXER IC 74155

AIM:

To demonstrate the use of MUX IC 74151, Decoder/Demux IC 74155.

OBJECTIVE:

- Students will study about multiplexer IC 74151.

COMPONENTS AND EQUIPMENTS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1.	8 : 1 MUX	IC 74151	1
2.	1 : 4 DEMUX	IC 74155	1
3.	IC TRAINER KIT	-	1
4.	Connecting wires	as required	

PRINCIPLE:

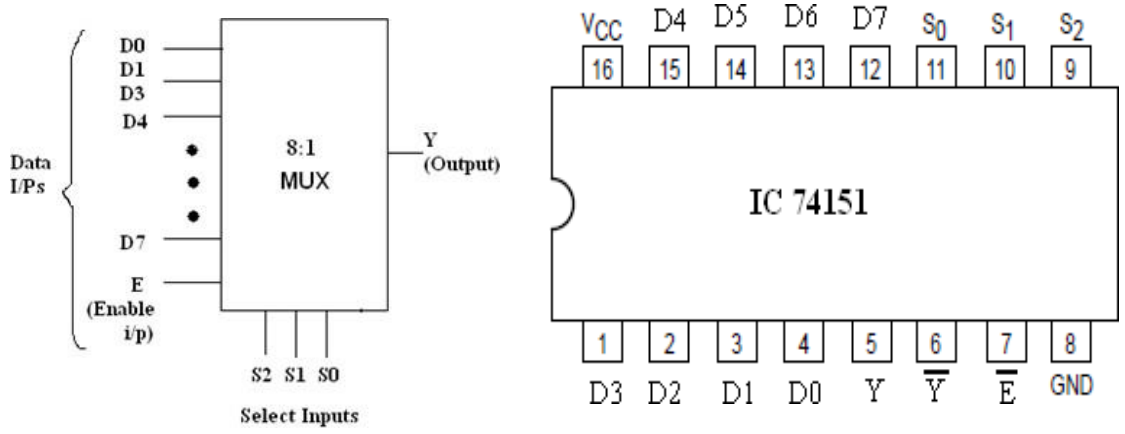
In most of the electronic systems, the digital data is available on more than one line. It is necessary to route this data over a single line. Under such circumstances we require a circuit which selects one of the many inputs at a time. Multiplexer improves the reliability of the digital system because it reduces the number of external wired connections.

An 8:1 Multiplexer has eight data inputs D0 to D7, three select inputs S0 to S2, an enable input and one output. Depending upon the digital code applied at the select inputs one out of n data input is selected & transmitted to a single o/p channel. Normally strobe (G) input is incorporated which is generally active low which enables the multiplexer when it is LOW. Strobe i/p helps in cascading.

IC 74151A is an 8: 1 multiplexer which provides two complementary outputs Y & $\overline{Y}$ The o/p Y is same as the selected i/p & is its complement.

CIRCUIT DIAGRAM:

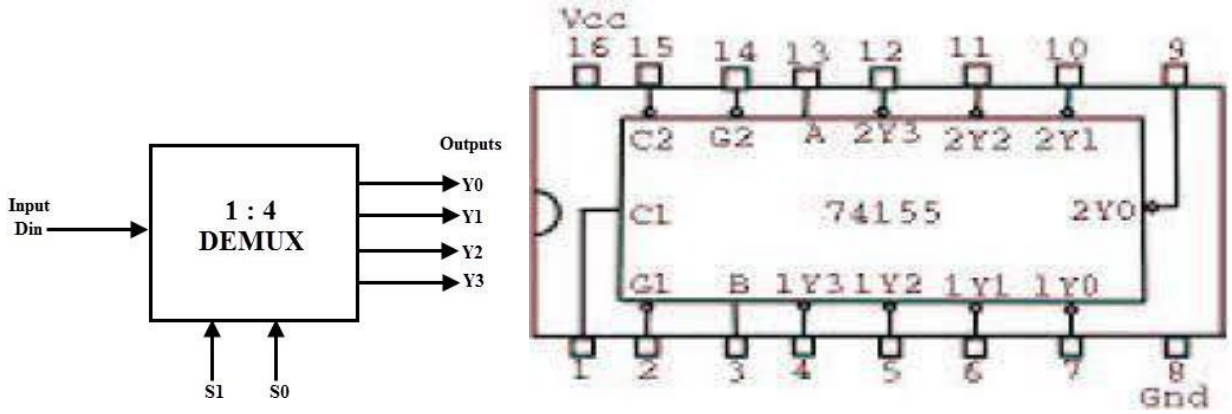
MULTIPLEXER IC 74151



[Note: Pin No.7 Enable pin is ACTIVE LOW input. So should be 0 to enable the IC].

ENABLE	SELECT INPUTS			OUTPUT
E (ACTIVE LOW)	S2	S1	S0	Y
1	X	X	X	0
0	0	0	0	D0
0	0	0	1	D1
0	0	1	0	D2
0	0	1	1	D3
0	1	0	0	D4
0	1	0	1	D5
0	1	1	0	D6
0	1	1	1	D7

DEMULTIPLEXER IC 74155



Data Input	Select Inputs		Outputs			
D	S_1	S_0	Y_3	Y_2	Y_1	Y_0
D	0	0	0	0	0	D
D	0	1	0	0	D	0
D	1	0	0	D	0	0
D	1	1	D	0	0	0

PROCEDURE:

1. Test the IC74151.
2. Connections are made as per circuit diagram.
3. Keep Enable pin HIGH to see that the output is always 0 for all input combinations.
4. Keep Enable pin LOW and verify the truth table for various combinations of inputs.

RESULT:

Studied about 8x1 MUX 74151 and verified its truth table.

Inference :

1. 74151 can be used to switch any of its 8 inputs to a single output line.

EXPERIMENT NO. 8

SR FLIP FLOP USING NAND GATES

AIM:

Construct SR flip flop using NAND gates and convert it into D flip flop.

COMPONENTS AND EQUIPMENTS REQUIRED:

Sl.No.	COMPONENT	SPECIFICATION	QTY.
1.	NAND GATE	IC 7400	2
2	NOT GATE	IC 7404	1
3.	IC TRAINER KIT	-	1
4.	Connecting wires	as required	

PRINCIPLE:

An SR Flip-Flop can be considered as a basic one-bit memory device that has two inputs, one which will "SET" the device and another which will "RESET" the device and an output Q that will be either at a logic level "1" or logic "0" depending upon this Set/Reset condition. A basic NAND Gate SR flip flop circuit provides feedback from its outputs to its inputs and is commonly used in memory circuits to store data bits. The term "Flip-flop" relates to the actual operation of the device, as it can be "Flipped" into one logic state or "Flopped" back into another. Flipflop is basically a Bistable Multivibrator.

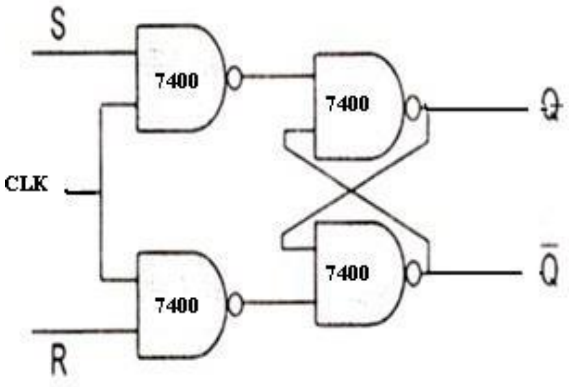
The symbol for a JK Flip-flop is similar to that of an SR flipflop. Both the S and the R inputs of the previous SR bistable have been replaced by two inputs called the J and K inputs, respectively. The two 2-input NAND gates of the gated SR bistable have now been replaced by two 3-input AND gates with the third input of each gate connected to the outputs Q and $\overline{Q}$.

This cross coupling of the SR Flip-flop allows the previously invalid condition of S = "1" and R = "1" state to be usefully used to turn it into a "Toggle action" as the two inputs are now interlocked. If the circuit is "Set" the J input is inhibited by the "0" status of the Q through the lower AND gate. If the circuit is "Reset" the K input is inhibited by the "0" status of Q through the upper AND gate. When both inputs J and K are equal to logic "1", the JK flip-flop changes state and the truth table for this is given below.

Although this circuit is an improvement on the clocked SR flip-flop it still suffers from timing problems called “race” .Consider clock=1 and J=K=1. This will cause the output to complement again and again. This complement operation continues until the Clock pulse goes back to 0. This undesirable behavior can be eliminated by Edge triggering of JK flip-flop or by using Master slave JK Flip-flops. To avoid this the timing pulse period must be kept as short as possible (high frequency).

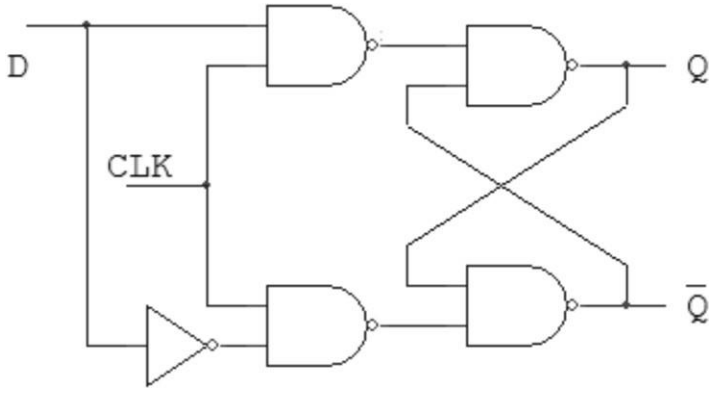
LOGIC DIAGRAM& TRUTH TABLE:

i) SR FLIP-FLOP



CLK	S	R	Qn+1
0	X	X	Qn **(**No Change)
1	0	0	No Change
1	0	1	0
1	1	0	1
1	1	1	Invalid

ii) D FLIP-FLOP



CLK	D	Qn+1
0	X	Qn (No Change)
1	0	0
1	1	1

PROCEDURE:

1. Connections are made as per circuit diagram.
2. Verify the truth table for various combinations of inputs.

RESULT

Constructed SR flip flop using NAND gates and convert it into D flip flop.

Inference :

1. RS flip flop can be converted to D Flip flop just by connecting R through a NOT gate to S.